

TaskFlow — Flutter Developer Technical Assignment (Mock Data Edition)

Problem Brief

Build the mobile application for TaskFlow, a lightweight project management system where users belong to organizations, create and manage projects, view and update tasks, assign work, and receive task-related notifications.

This assignment evaluates UI, architecture, and state management — not backend integration. You will build against local **mock JSON data** (provided) instead of a live REST API. Treat the mock data layer as if it were a real API: your networking/repository layer should be structured so it *could* be swapped for real HTTP calls later with minimal changes elsewhere in the app.

The solution should demonstrate clean Flutter architecture, state management, simulated authentication, a realistic data layer, responsive UI, error handling, and testing.

Technology Requirements

Category	Requirement
Framework	Flutter
Language	Dart
Platforms	Android required; iOS optional
Architecture	Clean / layered architecture preferred
State Management	Riverpod / Bloc / Cubit / Provider / equivalent
Data Source	Local mock JSON files (provided) — no real network calls required
Local Storage	SharedPreferences / Hive / Isar / Secure Storage / equivalent
Authentication	Simulated JWT-style authentication (mocked, see below)
Testing	Flutter unit/widget/integration tests
Documentation	README + architecture documentation

Not Allowed

- Calling a real/live backend or third-party API
- Firebase as the primary data source
- Backend-as-a-Service solutions
- Putting the entire application in a single file
- Skipping a proper data/repository layer by reading JSON directly in widgets

Provided

A single **TaskFlow-MockData.json** file is provided, containing all entities as top-level keys:

- **organizations**
- **users**
- **org_members** (role per user per org: **org_admin** / **member**)
- **projects**
- **tasks**
- **comments**
- **notifications**
- **auth_mock** (test credentials + mock token response)

Bundle this file as a single Flutter asset and read it through a repository/data-source layer — **not** directly in UI widgets. Parse each top-level key into its own model collection in your data layer.

Task 01 — Project Setup & Architecture

Create a scalable Flutter application structure.

Requirements

- Use a clear, scalable folder structure
- Separate presentation, business logic, and data-access responsibilities
- Keep widgets focused and reusable; avoid god classes
- Separate the "data source" (mock JSON reader) from the "repository" layer, so the repository interface would not need to change if it were later backed by real HTTP calls
- Separate models/entities from UI models where appropriate
- Use dependency injection or another clean dependency-management approach

Evaluation Focus: separation of concerns, maintainability, reusability, dependency management, scalability, readability.

Task 02 — Simulated Authentication

Implement authentication using the provided mock credentials and mock token response — **no real network call required**.

Required Screens

- Login
- Register (can simulate success without persisting new users anywhere but local state)
- Splash / Session Check

Requirements

- Validate login against `auth_mock.json` test credentials
- Client-side input validation with meaningful error messages
- On successful login, store the mock `access_token` / `refresh_token` pair securely (Secure Storage or equivalent)
- Simulate access token expiry after 15 minutes (use the `access_token_expires_in_seconds` value) and demonstrate a mock refresh flow that issues a new token
- Logout clears local session state and blocks access to authenticated screens afterward

Security Requirements

- Do not store passwords locally
- Do not log tokens
- Do not hardcode the mock credentials directly inside UI widgets — load them through the data layer

Bonus

- Biometric unlock for an existing authenticated session
 - Automatic session timeout after prolonged inactivity
-

Task 03 — Projects

Implement project management functionality against the mock data layer.

Required Screens

- Project List — name, description, task count, pull-to-refresh, loading/empty/error states
- Project Details — task summary counts grouped by status, list of project tasks

Requirements

- Read projects from the local mock data layer, scoped to the logged-in user's `org_id`
- Create / Edit / Delete project (updates local in-memory/local-storage state — no persistence to a real backend required)
- Confirm before destructive actions
- Refresh view after mutations

Authorization

- Only `org_admin` should be able to delete projects or manage members
 - The UI should not rely solely on hiding buttons — a non-admin attempting an admin action (e.g., via a direct deep link) should still be blocked in the business-logic layer, simulating what a real backend would enforce
-

Task 04 — Tasks

Implement complete task management against the mock data layer.

Task List

- Title, priority, status, assignee, due date
- Filters: status, priority, assignee, due-date range

Requirements

- View, create, edit, delete tasks (mock/local state)
 - Update task status and priority
 - Assign / unassign a user
 - View task details
 - Loading, empty, and error states
 - Refresh after mutation
-

Task 05 — Task Assignment & Users

Requirements

- Display org members from `org_members.json` + `users.json`, filtered to the current org
- Assign / remove a user from a task
- Display current assignee
- Prevent assigning a user who does not belong to the current org (validate this in the business-logic layer, not just by filtering the picker list)
- Refresh task state after assignment

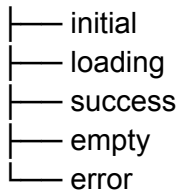
Task 06 — State Management

Use a proper state-management solution (Riverpod / Bloc / Cubit / Provider / GetX / other, well-justified).

Requirements

- Handle Initial → Loading → Success → Empty → Error consistently, e.g.:

TaskListState



- Avoid excessive `setState`, business logic inside widgets, or data access directly from UI widgets

Task 07 — Data Layer

Implement a reusable data-access layer reading from the mock JSON assets.

Requirements

- Centralized data-source class(es) for reading/writing mock JSON
- A repository layer with an interface that would not need to change if swapped for real HTTP later
- Request/response-style models (even though there's no real request)
- JSON serialization/deserialization
- Simulated error states: implement a way to force/simulate at least a few realistic conditions (e.g., "simulated 404 — task not found", "simulated network timeout", "simulated validation error") so error-handling UI can be demonstrated and tested. This can be a debug toggle, a specific mock ID that triggers an error, or a similar mechanism — document how to trigger it in the README.

Bonus

- Artificial network delay (e.g., 300–800ms) to make loading states demonstrable
 - Request cancellation
-

Task 08 — Offline Awareness (Simulated)

Requirements

- Persist last successfully loaded project/task data locally
- Since there is no real network, simulate a "connectivity toggle" (e.g., a debug switch or airplane-mode detection) that puts the app into an offline state
- When "offline": don't crash, show an appropriate offline message, preserve already-loaded data, allow retry
- Clearly indicate when displayed data may be stale

Bonus: offline-first task updates using a local pending-operations queue that "syncs" once back online.

Task 09 — UI/UX Requirements

Required

- Responsive layouts, consistent spacing/typography, reusable components
- Form validation, loading indicators, empty states, error states
- Confirmation dialogs for destructive actions
- Pull-to-refresh, proper navigation and back-navigation handling

Required Screens (minimum) Splash, Login, Register, Home/Dashboard, Projects, Project Details, Task List, Task Details, Create/Edit Task, Profile/Settings

Bonus: dark mode, tablet layout, animations, skeleton loading, accessibility, i18n.

Task 10 — Notifications (Bonus, not required)

The mock data includes `notifications.json` simulating task-assignment events.

- Optional: build a notification/inbox screen listing these
 - Optional: tapping a notification navigates to the relevant task
 - Not required for a passing submission — treat as bonus polish
-

Task 11 — Testing

Unit Tests (minimum)

- Session/auth logic (including simulated token refresh)

- Task filtering logic
- Validation logic
- State-management/business logic

Widget Tests

- Login form validation
- Task list rendering (loading / empty / error / success)
- Task status update UI

Integration Tests (minimum)

- Login flow (against mock credentials)
- Project listing
- Task listing
- Create/update task
- Task assignment

Tests should not depend on execution order or a real network connection — mock the data layer where appropriate.

Bonus: golden tests, code coverage report.

Task 12 — Documentation

README must include

- Project overview and architecture explanation
 - Folder structure
 - State-management approach
 - Mock data layer approach (how it's structured, how it simulates errors/delay/offline)
 - Auth/token flow (simulated)
 - Local setup, Flutter/Dart version, how to run, how to test, how to build APK
 - **How to trigger each simulated error/offline state** (for reviewer testing)
 - Known limitations and technical decisions/trade-offs
-

Task 13 — Build & Production Readiness

Requirements

- `flutter pub get` works successfully
- App runs without manual source-code modifications
- Debug build works; Release APK can be generated
- No committed secrets, no unnecessary debug logging in release builds

- No obvious memory leaks

Required Commands (document in README)

```
flutter pub get
flutter run
flutter test
flutter build apk --release
```

Submission Requirements

1. **GitHub Repository** — complete Flutter source, README.md, tests, the `mock_data/` assets, clean/meaningful commit history (not one giant initial commit)
 2. **Architecture Document** — app architecture, state management, data layer, simulated auth flow, local storage, error handling, navigation, key decisions (a simple diagram is encouraged)
 3. **Screen Recording (5–10 min)** — login, project listing, project details, task listing, task creation/editing, task assignment, status/priority update, simulated error/offline handling, logout — plus a brief explanation of architecture and decisions
 4. **Setup Instructions** — Flutter/Dart version, how to run, how to test, how to build release APK
 5. **Test Credentials** — provide the mock credentials for Org A (admin + member) and Org B (admin + member) from `auth_mock.json`, so reviewers can test role-based behavior
-

Evaluation Focus

Reviewers will primarily evaluate:

- Clean Flutter architecture and separation of concerns
- State-management quality
- Data layer design (would it survive being swapped for a real API?)
- Simulated authentication and token handling
- Secure local storage
- Error handling and loading/empty/error states
- UI consistency and responsive design
- Testing quality
- Documentation clarity

What we are *not* testing: real network integration, backend correctness, or production-scale data handling. The mock data layer is intentional — candidates should not attempt to stand up a real backend for this assignment.

Candidate Submission Checklist

- ☐ GitHub repository submitted with clean commit history
- ☐ Flutter project builds successfully
- ☐ README + architecture document included
- ☐ Mock data layer implemented (not hardcoded in widgets)
- ☐ Simulated authentication implemented
- ☐ Secure token storage implemented
- ☐ Projects and Tasks implemented against mock data
- ☐ Task assignment implemented
- ☐ Loading/empty/error states implemented
- ☐ Simulated error/offline states documented and demonstrable
- ☐ Automated tests included
- ☐ Test credentials documented
- ☐ Release APK can be generated
- ☐ Screen recording submitted
- ☐ Known limitations documented